코드 실행 환경 분석

1. 코드 실행 환경 개요

Open R1의 코드 실행 환경은 다양한 프로그래밍 언어와 문제 유형을 지원하는 샌드박스 시스템입니다. 안전하고 확장 가능한 코드 평가를 위해 여러 제공자를 통합하여 사용합니다.

1.1 지원 제공자

- **E2B**: Python 특화 클라우드 샌드박스
- **MorphCloud**: 다중 언어 지원 클라우드 샌드박스
- **Piston**: 경쟁 프로그래밍 문제 실행 환경
- **Local**: 로컬 실행 환경 (개발/테스트용)

1.2 시스템 아키텍처

```
코드 생성 → 코드 추출 → 실행 제공자 → 샌드박스 → 결과 평가 → 보상 계산
```

2. 코드 실행 제공자 시스템 (`src/open_r1/utils/code_providers.py`)

2.1 추상 기본 클래스

```
class CodeExecutionProvider(abc.ABC):
"""코드 실행 제공자의 추상 기본 클래스"""
```

```
@abc.abstractmethod
def  execute_scripts(self, scripts: List[str], languages: List[str]) -> List[flo
"""여러 스크립트를 실행하고 보상 값을 반환
Args:
scripts: 실행할 코드 스크립트 목록
languages: 각 스크립트의 프로그래밍 언어
Returns:
스크립트별 보상 값 목록 (float)
"""
pass
```

2.2 E2B 제공자

```
class E2BProvider(CodeExecutionProvider):
"""E2B 샌드박스를 사용한 코드 실행 제공자"""

def  __init__(self, num_parallel: int = 2, e2b_router_url: Optional[str] = None)
"""E2B 제공자 초기화
Args:
num_parallel: 병렬 샌드박스 수
e2b_router_url: E2B 라우터 URL (라우터 모드 사용 시)
"""
```

```
if not is_e2b_available():
raise ImportError(
"E2B is not available. Please install with "
"`pip install e2b-code-interpreter` and add API key to .env file."
)
self.num_parallel = num_parallel
self.e2b_router_url = e2b_router_url


def  execute_scripts(self, scripts: List[str], languages: List[str]) -> List[flo
"""E2B 샌드박스를 사용하여 스크립트 실행"""
if  self.e2b_router_url is not None:
# 라우터 모드: 배치 처리
routed_sandbox = RoutedSandbox(router_url=self.e2b_router_url)
executions = routed_sandbox.run_code(
scripts=scripts,
languages=languages,
timeout=30,
request_timeout=28,
)
rewards = []
for execution in executions:
try:
if execution.results and len(execution.results) > 0:
```

```
# 마지막 결과의 텍스트 출력을 보상으로 사용

result_text = execution.results[-1].text

reward = float(result_text.strip())

else:

reward = 0.0

except (ValueError, AttributeError):

reward = 0.0

rewards.append(reward)

return rewards

else:

# 직접 모드: 병렬 처리

return asyncio.run(self._execute_parallel(scripts, languages))

async def _execute_parallel(self, scripts: List[str], languages: List[str]) ->

"""병렬 실행을 위한 비동기 함수"""

semaphore = asyncio.Semaphore(self.num_parallel)

async def execute_single(script: str, language: str) -> float:

async with semaphore:

async with AsyncSandbox() as sandbox:

try:

execution = await sandbox.run_code(script, language=language)

if execution.results and len(execution.results) > 0:

result_text = execution.results[-1].text

return float(result_text.strip())
```

```
else:
return  0.0
except Exception:
return  0.0
tasks = [execute_single(script, lang) for script, lang in  zip(scripts, language
return  await asyncio.gather(*tasks)
```

2.3 MorphCloud 제공자

```
class MorphProvider(CodeExecutionProvider):
"""MorphCloud를 사용한 다중 언어 코드 실행 제공자"""

def  __init__(self, num_parallel: int = 2, morph_router_url: Optional[str] = Non
"""MorphCloud 제공자 초기화"""
if not is_morph_available():
raise ImportError(
"MorphCloud is not available. Please install with "
"`pip install morphcloud` and add API key to .env file."
)
self.num_parallel = num_parallel
self.morph_router_url = morph_router_url
```

```
def  execute_scripts(self, scripts: List[str], languages: List[str]) -> List[flo

"""MorphCloud를 사용하여 스크립트 실행"""

if  self.morph_router_url is not None:

# 라우터 모드 사용

routed_morph = RoutedMorphSandbox(router_url=self.morph_router_url)

results = routed_morph.run_code(

scripts=scripts,

languages=languages,

timeout=30,

request_timeout=28,

)

rewards = []

for result in results:

try:

if result.text:

reward = float(result.text.strip())

else:

reward = 0.0

except (ValueError, AttributeError):

reward = 0.0

rewards.append(reward)

return rewards

else:
```

```
# 직접 모드 사용
return asyncio.run(self._execute_parallel(scripts, languages))
```

2.4 제공자 팩토리

```
def  get_provider(
provider_type: str = "e2b",
num_parallel: int = 2,
**kwargs
) -> CodeExecutionProvider:
"""코드 실행 제공자 팩토리 함수
Args:
provider_type: 제공자 유형 ("e2b", "morph", "local")
num_parallel: 병렬 실행 수
**kwargs: 제공자별 추가 매개변수
Returns:
CodeExecutionProvider: 설정된 코드 실행 제공자
"""
if provider_type == "e2b":
return E2BProvider(
num_parallel=num_parallel,
e2b_router_url=kwargs.get("e2b_router_url")
)
```

```
elif provider_type == "morph":
return MorphProvider(
num_parallel=num_parallel,
morph_router_url=kwargs.get("morph_router_url")
)
elif provider_type == "local":
return LocalProvider(num_parallel=num_parallel)
else:
raise ValueError(f"Unknown provider type: {provider_type}")
```

3. E2B 라우터 서비스 (`scripts/e2b_router.py`)

3.1 라우터 개요

E2B 라우터는 여러 훈련 프로세스에서 E2B API 호출을 중앙화하여 속도 제한을 관리하고 부하를 분산하는 FastAPI 서비스입니다.

3.2 데이터 모델

```
class BatchRequest(BaseModel):
"""배치 처리 요청 모델"""
scripts: list[str] # 실행할 스크립트 목록
languages: list[str] # 각 스크립트의 언어
timeout: int # 스크립트별 타임아웃 (초)
request_timeout: int # 전체 요청 타임아웃 (초)
```

```
class ScriptResult(BaseModel):
"""스크립트 실행 결과 모델"""
execution: Optional[Execution] # E2B 실행 결과
exception_str: Optional[str] # 예외 메시지
model_config = ConfigDict(arbitrary_types_allowed=True)
```

3.3 FastAPI 애플리케이션

```
def create_app(args):
"""E2B 라우터 FastAPI 애플리케이션 생성"""
app = FastAPI()
# 동시 샌드박스 수 제한을 위한 세마포어
app.state.sandbox_semaphore = asyncio.Semaphore(args.max_num_sandboxes)


@app.get("/health")
async def health():
"""헬스 체크 엔드포인트"""
return {"status": "ok"}


@app.post("/execute_batch")
async def execute_batch(batch: BatchRequest, request: Request):
"""배치 스크립트 실행 엔드포인트"""
```

```
semaphore = request.app.state.sandbox_semaphore

async  def  run_script(script: str, language: str) -> ScriptResult:

"""단일 스크립트 실행"""

async  with semaphore:

sandbox = None

try:

# E2B 샌드박스 생성 및 실행

sandbox = await AsyncSandbox.acreate()

# 타임아웃과 함께 스크립트 실행

execution = await asyncio.wait_for(

sandbox.run_code(script, language=language),

timeout=batch.timeout

)

return ScriptResult(execution=execution, exception_str=None)

except asyncio.TimeoutError:

return ScriptResult(

execution=None,

exception_str=f"Script execution timed out after {batch.timeout}s"

)

except Exception as e:

return ScriptResult(

execution=None,

exception_str=str(e)
```

```
)
finally:
if sandbox:
await sandbox.aclose()
# 모든 스크립트를 병렬로 실행
tasks = [
run_script(script, language)
for script, language in  zip(batch.scripts, batch.languages)
]
try:
results = await asyncio.wait_for(
asyncio.gather(*tasks, return_exceptions=True),
timeout=batch.request_timeout
)
return results
except asyncio.TimeoutError:
return [
ScriptResult(
execution=None,
exception_str=f"Batch request timed out after {batch.request_timeout}s"
)
for _ in batch.scripts
]
```

```
return app
```

3.4 라우터 실행

```
# E2B 라우터 서버 시작
python scripts/e2b_router.py \
--host 0.0.0.0 \
--port 8001 \
--max-num-sandboxes 10

# Slurm에서 라우터 시작
sbatch slurm/e2b_router.slurm
```

4. MorphCloud 라우터 서비스 (`scripts/morph_router.py`)

4.1 MorphCloud 특화 기능

```
def create_app(args):
"""MorphCloud 라우터 FastAPI 애플리케이션 생성"""
app = FastAPI()
# MorphCloud 클라이언트 초기화
from morphcloud.api import MorphCloudClient
from morphcloud.sandbox import Sandbox
```

```
app.state.client = MorphCloudClient(api_key=args.api_key)

app.state.Sandbox = Sandbox

app.state.sandbox_semaphore = asyncio.Semaphore(args.max_num_sandboxes)



@app.post("/execute_batch")

async  def  execute_batch(batch: BatchRequest, request: Request):

"""MorphCloud 배치 실행"""

semaphore = request.app.state.sandbox_semaphore

client = request.app.state.client

Sandbox = request.app.state.Sandbox

async  def  run_script(script: str, language: str) -> ScriptResult:

"""MorphCloud에서 단일 스크립트 실행"""

sandbox = None

sandbox_id = "unknown"

async  with semaphore:

try:

# MorphCloud 샌드박스 생성

sandbox = await client.sandboxes.create(

template="ubuntu-22.04",

timeout=batch.timeout + 10

)

sandbox_id = sandbox.id

# 언어별 실행 명령 설정
```

```
if language == "python":
command = f"python3 -c '{script}'"
elif language == "javascript":
command = f"node -e '{script}'"
elif language == "cpp":
# C++ 컴파일 및 실행
command = f"""
echo '{script}' > solution.cpp &&
g++ -o solution solution.cpp &&
./solution
"""
else:
raise ValueError(f"Unsupported language: {language}")
# 명령 실행
result = await asyncio.wait_for(
sandbox.exec(command),
timeout=batch.timeout
)
return ScriptResult(text=result.stdout, exception_str=None)
except asyncio.TimeoutError:
return ScriptResult(
text=None,
exception_str=f"Execution timed out after {batch.timeout}s"
```

```
)
except Exception as e:
return ScriptResult(
text=None,
exception_str=f"Error in sandbox {sandbox_id}: {str(e)}"
)
finally:
if sandbox:
try:
await sandbox.delete()
except Exception:
pass
# 병렬 실행
tasks = [
run_script(script, language)
for script, language in  zip(batch.scripts, batch.languages)
]
try:
results = await asyncio.wait_for(
asyncio.gather(*tasks, return_exceptions=True),
timeout=batch.request_timeout
)
return results
```

```
except asyncio.TimeoutError:
return [
ScriptResult(
text=None,
exception_str=f"Batch timed out after {batch.request_timeout}s"
)
for _ in batch.scripts
]


return app
```

5. 경쟁 프로그래밍 환경

5.1 Piston 클라이언트 (`src/open_r1/utils/competitive_programming/piston_client.py`)

```
class PistonClient:
"""여러 Piston 워커에 자동 부하 분산하는 클라이언트"""


def __init__(
self,
base_endpoint: str | list[str] = "http://localhost:3223/api/v2",
session=None,
```

```
max_requests_per_endpoint=1,
):
"""Piston 클라이언트 초기화
Args:
base_endpoint: Piston 엔드포인트 URL(들)
session: aiohttp 세션 (선택적)
max_requests_per_endpoint: 엔드포인트당 최대 동시 요청 수
"""
self.max_requests_per_endpoint = max_requests_per_endpoint
self.base_endpoints = [base_endpoint] if  isinstance(base_endpoint, str) else ba
if  len(self.base_endpoints) == 0:
raise ValueError("No Piston endpoints provided")
# 엔드포인트별 토큰 큐 (부하 분산용)
self.endpoint_tokens = asyncio.Queue(
maxsize=max_requests_per_endpoint * len(self.base_endpoints)
)
# 각 엔드포인트에 대해 토큰 생성
for _ in  range(max_requests_per_endpoint):
for base_endpoint in  self.base_endpoints:
self.endpoint_tokens.put_nowait(base_endpoint)
self._endpoint_failures = Counter()
self._unhealthy_endpoints = set()
```

```
async def execute_code(
self,
language: str,
source_code: str,
stdin: str = "",
timeout: int = 30
) -> dict:
"""Piston에서 코드 실행
Args:
language: 프로그래밍 언어 ("cpp", "python", "javascript" 등)
source_code: 소스 코드
stdin: 표준 입력
timeout: 실행 타임아웃
Returns:
실행 결과 딕셔너리
"""
endpoint = await self._wait_for_endpoint()
try:
payload = {
"language": language,
"version": "*", # 최신 버전 사용
"files": [
{
```

```
"name": f"main.{self._get_extension(language)}",
"content": source_code
}
],
"stdin": stdin,
"compile_timeout": timeout * 1000, # ms 단위
"run_timeout": timeout * 1000,
"compile_memory_limit": 128 * 1024 * 1024, # 128MB
"run_memory_limit": 256 * 1024 * 1024, # 256MB
}
async with self.session.post(
f"{endpoint}/execute",
json=payload,
timeout=aiohttp.ClientTimeout(total=timeout + 10)
) as response:
if response.status == 200:
result = await response.json()
return result
else:
raise PistonError(f"HTTP {response.status}: {await response.text()}")
except Exception as e:
await self._handle_endpoint_error(endpoint, e)
raise
```

```
finally:
    await self._release_endpoint(endpoint)


def _get_extension(self, language: str) -> str:
    """언어별 파일 확장자 반환"""
    extensions = {
        "cpp": "cpp",
        "c": "c",
        "python": "py",
        "javascript": "js",
        "java": "java",
        "rust": "rs",
        "go": "go"
    }
    return extensions.get(language, "txt")
```

5.2 IOI 문제 지원

```
# src/open_r1/utils/competitive_programming/ioi_scoring.py


@dataclass
class SubtaskResult:
    """IOI 서브태스크 결과"""
```

```
score: float = 0.0

max_score: float = 0.0

verdict: str = "WA"  # AC, WA, TLE, RTE, CE

test_results: List[dict] = field(default_factory=list)



async  def  score_subtask(

client: PistonClient,

problem_data: dict,

code: str,

test_batch_size: int = 1

) -> SubtaskResult:

"""IOI 서브태스크 채점

Args:

client: Piston 클라이언트

problem_data: 문제 데이터 (테스트 케이스 포함)

code: 제출할 C++ 코드

test_batch_size: 배치당 테스트 케이스 수

Returns:

SubtaskResult: 채점 결과

"""

if not code.strip():

return SubtaskResult()

test_cases = problem_data.get("test_cases", [])
```

```
if not test_cases:
return SubtaskResult()
total_score = 0.0
max_score = len(test_cases)
test_results = []
# 배치 단위로 테스트 케이스 처리
for i in  range(0, len(test_cases), test_batch_size):
batch = test_cases[i:i + test_batch_size]
# 배치 내 테스트 케이스들을 병렬 실행
tasks = []
for test_case in batch:
task = client.execute_code(
language="cpp",
source_code=code,
stdin=test_case["input"],
timeout=test_case.get("time_limit", 2)
)
tasks.append(task)
try:
results = await asyncio.gather(*tasks, return_exceptions=True)
for test_case, result in  zip(batch, results):
if  isinstance(result, Exception):
test_results.append({
```

```
"verdict": "RTE",
"score": 0.0,
"error": str(result)
})
continue
# 결과 검증
if result.get("run", {}).get("code") != 0:
verdict = "RTE"
score = 0.0
elif result.get("run", {}).get("signal"):
verdict = "RTE"
score = 0.0
else:
output = result.get("run", {}).get("stdout", "").strip()
expected = test_case["output"].strip()
if output == expected:
verdict = "AC"
score = 1.0
total_score += 1.0
else:
verdict = "WA"
score = 0.0
test_results.append({
```

```
"verdict": verdict,
"score": score,
"output": result.get("run", {}).get("stdout", ""),
"expected": test_case["output"]
})
# 첫 번째 실패 시 조기 종료 (최적화)
if score == 0.0 and test_batch_size == 1:
break
except Exception as e:
# 배치 전체 실패
for test_case in batch:
test_results.append({
"verdict": "RTE",
"score": 0.0,
"error": str(e)
})
final_score = total_score / max_score if max_score > 0  else  0.0
overall_verdict = "AC"  if final_score == 1.0  else  "WA"
return SubtaskResult(
score=final_score,
max_score=1.0,
verdict=overall_verdict,
test_results=test_results
```

```
)
```

5.3 CodeForces 문제 지원

```
# src/open_r1/utils/competitive_programming/cf_scoring.py

async def score_submission(
client: PistonClient,
problem_data: dict,
code: str,
test_batch_size: int = 1,
scoring_mode: str = "weighted_sum",
submission_language: str = None
) -> float:
"""CodeForces 제출 채점
Args:
client: Piston 클라이언트
problem_data: 문제 데이터
code: 제출 코드
test_batch_size: 배치 크기
scoring_mode: 채점 모드 ("pass_fail", "partial", "weighted_sum")
submission_language: 제출 언어
Returns:
```

```
점수 (0.0 ~ 1.0)
"""
if not code.strip():
return  0.0
# 테스트 케이스 로드
test_cases = load_test_cases(problem_data)
if not test_cases:
return  0.0
language = submission_language or detect_language(code)
if not language:
return  0.0
passed_tests = 0
total_tests = len(test_cases)
test_scores = []
# 배치 단위 실행
for i in  range(0, total_tests, test_batch_size):
batch = test_cases[i:i + test_batch_size]
tasks = []
for test_case in batch:
task = client.execute_code(
language=language,
source_code=code,
stdin=test_case["input"],
```

```
timeout=test_case.get("time_limit", 2)

)

tasks.append(task)

try:

results = await asyncio.gather(*tasks, return_exceptions=True)

for test_case, result in  zip(batch, results):

if  isinstance(result, Exception):

test_scores.append(0.0)

continue

# 실행 결과 검증

if (result.get("run", {}).get("code") == 0 and

not result.get("run", {}).get("signal")):

output = result.get("run", {}).get("stdout", "").strip()

expected = test_case["output"].strip()

if compare_outputs(output, expected):

test_scores.append(1.0)

passed_tests += 1

else:

test_scores.append(0.0)

else:

test_scores.append(0.0)

# 조기 종료 조건 (pass_fail 모드)

if scoring_mode == "pass_fail" and test_scores[-1] == 0.0:
```

```
return  0.0

except Exception:

# 배치 실패 시 0점 처리

test_scores.extend([0.0] * len(batch))

# 채점 모드별 점수 계산

if scoring_mode == "pass_fail":

return  1.0  if passed_tests == total_tests else  0.0

elif scoring_mode == "partial":

return passed_tests / total_tests

elif scoring_mode == "weighted_sum":

# 가중합 (초기 테스트가 더 중요)

weights = [1.0 / (i + 1) for i in  range(total_tests)]

weighted_score = sum(score * weight for score, weight in  zip(test_scores, weigh

total_weight = sum(weights)

return weighted_score / total_weight

else:

raise ValueError(f"Unknown scoring mode: {scoring_mode}")



def  compare_outputs(actual: str, expected: str) -> bool:

"""출력 비교 (공백 정규화 포함)"""

# 줄별로 비교하여 공백 차이 무시

actual_lines = [line.strip() for line in actual.split('\n') if line.strip()]

expected_lines = [line.strip() for line in expected.split('\n') if line.strip()]
```

```
return actual_lines == expected_lines
```

6. 환경 설정 및 배포

6.1 E2B 설정

```
# .env 파일 설정
E2B_API_KEY="e2b_your_api_key_here"

# 의존성 설치
pip install e2b-code-interpreter

# 라우터 서버 시작
python scripts/e2b_router.py --host 0.0.0.0 --port 8001 --max-num-sandboxe
```

6.2 MorphCloud 설정

```
# .env 파일 설정
MORPH_API_KEY="your_morph_api_key_here"

# 의존성 설치
pip install morphcloud
```

```
# 라우터 서버 시작
python scripts/morph_router.py --host 0.0.0.0 --port 8002 --max-num-sandbo
```

6.3 Piston 설정

```
# .env 파일 설정
PISTON_ENDPOINTS="http://piston1:3223/api/v2,http://piston2:3223/api/v2"
PISTON_MAX_REQUESTS_PER_ENDPOINT="2"

# Piston 서버 시작 (각 워커에서)
export PISTON_COMPILE_TIMEOUT=60000
export PISTON_RUN_TIMEOUT=60000
export PISTON_OUTPUT_MAX_SIZE=1000000000
export PISTON_MAX_FILE_SIZE=1000000000
export PISTON_DISABLE_NETWORKING=true
export PISTON_REPO_URL=https://github.com/guipenedo/piston/releases/download/pk

# Piston 서버 실행
node src/index.js
```

6.4 Slurm 통합

```
# E2B 라우터 Slurm 작업
```

```
sbatch slurm/e2b_router.slurm


# MorphCloud 라우터 Slurm 작업
sbatch slurm/morph_router.slurm


# Piston 워커들 시작
sbatch --array=1-4 slurm/piston/start_worker.slurm
```

7. 성능 최적화

7.1 동시성 튜닝

```
# 하드웨어별 최적 동시성 설정
def get_optimal_concurrency(provider_type: str, available_memory_gb: int) -> in
"""하드웨어 사양에 따른 최적 동시성 계산"""
if provider_type == "e2b":
# E2B는 API 제한이 있으므로 보수적으로 설정
if available_memory_gb >= 32:
return 8
elif available_memory_gb >= 16:
return 4
else:
return 2
```

```
elif provider_type == "morph":
# MorphCloud는 더 많은 동시성 허용
if available_memory_gb >= 64:
return 16
elif available_memory_gb >= 32:
return 8
else:
return 4
elif provider_type == "piston":
# Piston은 로컬 리소스에 따라 결정
return min(available_memory_gb // 2, 12)
return 2
```

7.2 캐싱 전략

```
from functools import lru_cache
import hashlib

class CachedCodeExecutor:
"""코드 실행 결과 캐싱"""
def __init__(self, provider: CodeExecutionProvider, cache_size: int = 1000):
self.provider = provider
self.cache = {}
```

```
self.cache_size = cache_size

def  _get_cache_key(self, script: str, language: str) -> str:

"""캐시 키 생성"""

content = f"{language}:{script}"

return hashlib.md5(content.encode()).hexdigest()

def  execute_scripts(self, scripts: List[str], languages: List[str]) -> List[flo

"""캐시를 사용한 스크립트 실행"""

results = []

uncached_indices = []

uncached_scripts = []

uncached_languages = []

# 캐시 확인

for i, (script, language) in  enumerate(zip(scripts, languages)):

cache_key = self._get_cache_key(script, language)

if cache_key in self.cache:

results.append(self.cache[cache_key])

else:

results.append(None) # 나중에 채울 자리 표시

uncached_indices.append(i)

uncached_scripts.append(script)

uncached_languages.append(language)

# 캐시되지 않은 스크립트 실행

if uncached_scripts:
```

```
uncached_results = self.provider.execute_scripts(
uncached_scripts, uncached_languages
)
# 결과를 캐시에 저장하고 결과 배열에 채움
for idx, result in  zip(uncached_indices, uncached_results):
cache_key = self._get_cache_key(scripts[idx], languages[idx])
# 캐시 크기 제한
if  len(self.cache) >= self.cache_size:
# LRU 방식으로 오래된 항목 제거
oldest_key = next(iter(self.cache))
del  self.cache[oldest_key]
self.cache[cache_key] = result
results[idx] = result
return results
```

7.3 오류 처리 및 재시도

```
import asyncio
import random
from typing import Optional


class RobustCodeExecutor:
"""견고한 코드 실행기 (재시도 및 오류 처리 포함)"""
```

```
def __init__(
self,
provider: CodeExecutionProvider,
max_retries: int = 3,
base_delay: float = 1.0,
max_delay: float = 60.0
):
self.provider = provider
self.max_retries = max_retries
self.base_delay = base_delay
self.max_delay = max_delay
async def execute_with_retry(
self,
scripts: List[str],
languages: List[str]
) -> List[Optional[float]]:
"""재시도 로직이 포함된 실행"""
for attempt in range(self.max_retries + 1):
try:
results = self.provider.execute_scripts(scripts, languages)
# None 결과가 있는지 확인 (부분 실패)
if any(result is None for result in results):
if attempt < self.max_retries:
```

```
# 부분 실패 시 재시도
delay = min(
self.base_delay * (2 ** attempt) + random.uniform(0, 1),
self.max_delay
)
await asyncio.sleep(delay)
continue
return results
except Exception as e:
if attempt < self.max_retries:
# 지수 백오프로 재시도
delay = min(
self.base_delay * (2 ** attempt) + random.uniform(0, 1),
self.max_delay
)
print(f"Execution failed (attempt {attempt + 1}): {e}")
print(f"Retrying in {delay:.2f} seconds...")
await asyncio.sleep(delay)
else:
print(f"All {self.max_retries + 1} attempts failed: {e}")
# 모든 재시도 실패 시 None 반환
return [None] * len(scripts)
```

```
return [None] * len(scripts)
```

8. 모니터링 및 디버깅

8.1 실행 통계 수집

```
import time

from collections import defaultdict, Counter

from dataclasses import dataclass

from typing import Dict, List


@dataclass

class ExecutionStats:

"""실행 통계"""

total_executions: int = 0

successful_executions: int = 0

failed_executions: int = 0

total_time: float = 0.0

avg_time: float = 0.0

language_stats: Dict[str, int] = None

error_types: Dict[str, int] = None

def __post_init__(self):

if  self.language_stats is None:
```

```
self.language_stats = defaultdict(int)

if  self.error_types is None:

self.error_types = Counter()



class MonitoredCodeExecutor:

"""모니터링 기능이 포함된 코드 실행기"""

def  __init__(self, provider: CodeExecutionProvider):

self.provider = provider

self.stats = ExecutionStats()

def  execute_scripts(self, scripts: List[str], languages: List[str]) -> List[flo

"""모니터링과 함께 스크립트 실행"""

start_time = time.time()

try:

results = self.provider.execute_scripts(scripts, languages)

# 통계 업데이트

execution_time = time.time() - start_time

self.stats.total_executions += len(scripts)

self.stats.total_time += execution_time

for language in languages:

self.stats.language_stats[language] += 1

successful = sum(1  for r in results if r is  not None and r > 0)

failed = len(results) - successful

self.stats.successful_executions += successful
```

```
self.stats.failed_executions += failed

# 평균 시간 계산

if  self.stats.total_executions > 0:

self.stats.avg_time = self.stats.total_time / self.stats.total_executions

return results

except Exception as e:

# 오류 통계 업데이트

error_type = type(e).__name__

self.stats.error_types[error_type] += 1

self.stats.failed_executions += len(scripts)

print(f"Execution error: {e}")

return [None] * len(scripts)

def  get_stats(self) -> ExecutionStats:

"""현재 통계 반환"""

return  self.stats

def  reset_stats(self):

"""통계 초기화"""

self.stats = ExecutionStats()
```

8.2 로깅 및 디버깅

```
import logging

from datetime import datetime
```

```
# 로깅 설정
logging.basicConfig(
level=logging.INFO,
format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
handlers=[
logging.FileHandler('code_execution.log'),
logging.StreamHandler()
]
)

logger = logging.getLogger(__name__)

class DebuggingCodeExecutor:
"""디버깅 기능이 포함된 코드 실행기"""
def __init__(self, provider: CodeExecutionProvider, debug: bool = False):
self.provider = provider
self.debug = debug
def execute_scripts(self, scripts: List[str], languages: List[str]) -> List[flo
"""디버깅 정보와 함께 스크립트 실행"""
if self.debug:
logger.info(f"Executing {len(scripts)} scripts")
for i, (script, lang) in enumerate(zip(scripts, languages)):
```

```
logger.debug(f"Script {i} ({lang}): {script[:100]}...")

start_time = datetime.now()

try:

results = self.provider.execute_scripts(scripts, languages)

execution_time = (datetime.now() - start_time).total_seconds()

if  self.debug:

logger.info(f"Execution completed in {execution_time:.2f}s")

for i, result in  enumerate(results):

if result is None:

logger.warning(f"Script {i} failed")

else:

logger.debug(f"Script {i} result: {result}")

return results

except Exception as e:

logger.error(f"Execution failed: {e}", exc_info=True)

return [None] * len(scripts)
```

코드 실행 환경은 Open R1의 강화학습 훈련에서 핵심적인 역할을 하며, 다양한 제공자와 최적화 기법을 통해 안전하고 효율적인 코드 평가를 제공합니다. 라우터 서비스를 통한 부하 분산과 캐싱, 재시도 메커니즘을 통해 대규모 훈련에서도 안정적으로 동작합니다.